

Q3. Debugging Linear Search Code (Incorrect Loop Starting Index)

The following Linear Search implementation begins checking from the wrong index and may

skip a valid match at the start of the array. Carefully analyze the loop initialization and explain why the first element is ignored. Apply debugging to ensure that every element from

the beginning of the array is checked correctly. Optimize the implementation for simplicity and readability while preserving correct functionality. Analyze the time complexity of the corrected version and rewrite the final code with proper comments.

```
def linear_search(arr, key):  
    for i in range(1, len(arr)): # ERROR  
        if arr[i] == key:  
            return i  
    return -1
```

. Q3. Debugging Linear Search Code — Incorrect Loop Starting Index

1. The Buggy Code

```
python  
def linear_search(arr, key):  
    for i in range(1, len(arr)): # ERROR  
        if arr[i] == key:  
            return i  
    return -1
```

2. Analysis of the Error

Problem: The loop uses `range(1, len(arr))`.

In Python, `range(1, len(arr))` generates indices starting from 1 up to `len(arr) - 1`, meaning:

`i = 1, 2, 3, ..., len(arr) - 1`

This **completely skips index 0** — the **first element** of the array.

Why this is wrong:

- If the key (search target) is located at `arr[0]`, the loop never checks it because iteration begins at index 1.
- The function will incorrectly return `-1` (not found) even though the element exists at the start of the array.
- This is an **off-by-one error**, a very common bug in loop initialization.

Example demonstrating the bug:

```
python
arr = [10, 20, 30, 40]
key = 10
linear_search(arr, key)  # Should return 0, but returns -1 (BUG)
```

3. The Fix

Change the starting index of the range from `1` to `0`, so every element — including the first — is checked:

```
python
for i in range(0, len(arr)):  # FIXED — starts from index 0
```

This can be further simplified since `range(0, n)` is equivalent to `range(n)` in Python:

```
python
for i in range(len(arr)):  # simplified, same effect
```

4. Corrected and Optimized Code

```
python
def linear_search(arr, key):
    """
    Searches for 'key' in 'arr' using linear search.
    Checks every element starting from index 0.
    Returns the index of key if found, otherwise -1.
    """
    for i in range(len(arr)):  # FIX: starts from index 0, checks
all elements
        if arr[i] == key:
```

```

        return i                # key found, return its index
    return -1                   # key not found in the array

```

5. Time Complexity Analysis (Corrected Version)

Case	Condition	Time Complexity
Best Case	Key found at index 0	$O(1)$
Worst Case	Key found at last index, or not present	$O(n)$
Average Case	Key equally likely at any position	$O(n)$

- **Space Complexity:** $O(1)$ — no extra space used, search is done in-place.
- The fix does **not** change the asymptotic time complexity (still $O(n)$ worst case) — it only ensures **correctness**, since the original buggy version was already $O(n)$ but produced **wrong results** by ignoring index 0.

6. Summary

Aspect	Before (Buggy)	After (Fixed)
Loop range	<code>range(1, len(arr))</code>	<code>range(len(arr))</code>
Index 0 checked?	No (skipped)	Yes
Correctness	Incorrect for matches at index 0	Correct for all cases
Time Complexity	$O(n)$ (but wrong output)	$O(n)$ (correct output)